Engs 28

Embedded Systems

Day 3

What is the most interesting thing you have learned so far?

Nobody has responded yet.

Hang tight! Responses are coming in.

What is the muddiest point so far?

Nobody has responded yet.

Hang tight! Responses are coming in.

How useful was the "Explaining Blinky" HiTA activity?

Nobody has responded yet.

Hang tight! Responses are coming in.

Reminder: Late Lab Assignment Policy

- Each student has an account of 2 late days to be used at their discretion for lab assignments. If you wish to use one of your late days for a lab assignment, please indicate you are doing so in a submission comment.

- If you use a late day for your lab assignment and submit within 24 hours of the original deadline you will not be penalized for your late submission. Otherwise:

- Late lab assignments will be accepted for up to two days after they are due (with due date modified based on number of used late days) at the cost of a 10% penalty per day.

- After two days beyond the deadline lab assignments will not be accepted for credit.

Another Reminder: Labs and the Honor Code

- You will work alone on the labs unless we specifically say otherwise.
- Interaction, to a certain degree, among class members is encouraged when learning new things. But, as you write and debug your programs, be careful that in your conversations you are exchanging ideas, not code, and that the work you turn in is your own. Do not copy any portion of someone else's computer code or provide code to someone else. Copying another person's work — including reusing a design posted online or written by a student in another course — is a violation of the Honor Principle and will have serious consequences.
- Always cite your lab and study partners by giving their names on your papers, just as you would acknowledge any other source (such as a book or article) outside the usual course readings.

A Third Reminder: AI Use in ENGS 28

You are free to use the HiTA tool, integrated in Canvas, to get help with the assignments in this course. Please do not use any other GenAI tools unless explicitly invited to do so.

We will treat GenAI tools similarly to other resources you use. This means you MUST follow these four points:

- Give notice that you used the AI tool and how you used it in the comments of your code and your lab report.
- If you are basing your work on generated code, rigorously test and alter the program to suit the assignment and your understanding.
- You must understand any code you submit and be prepared to explain it to us.
- All comments in your code should be your own words.

Agenda

Code Review for Lab 1 code blinkyCNT.c

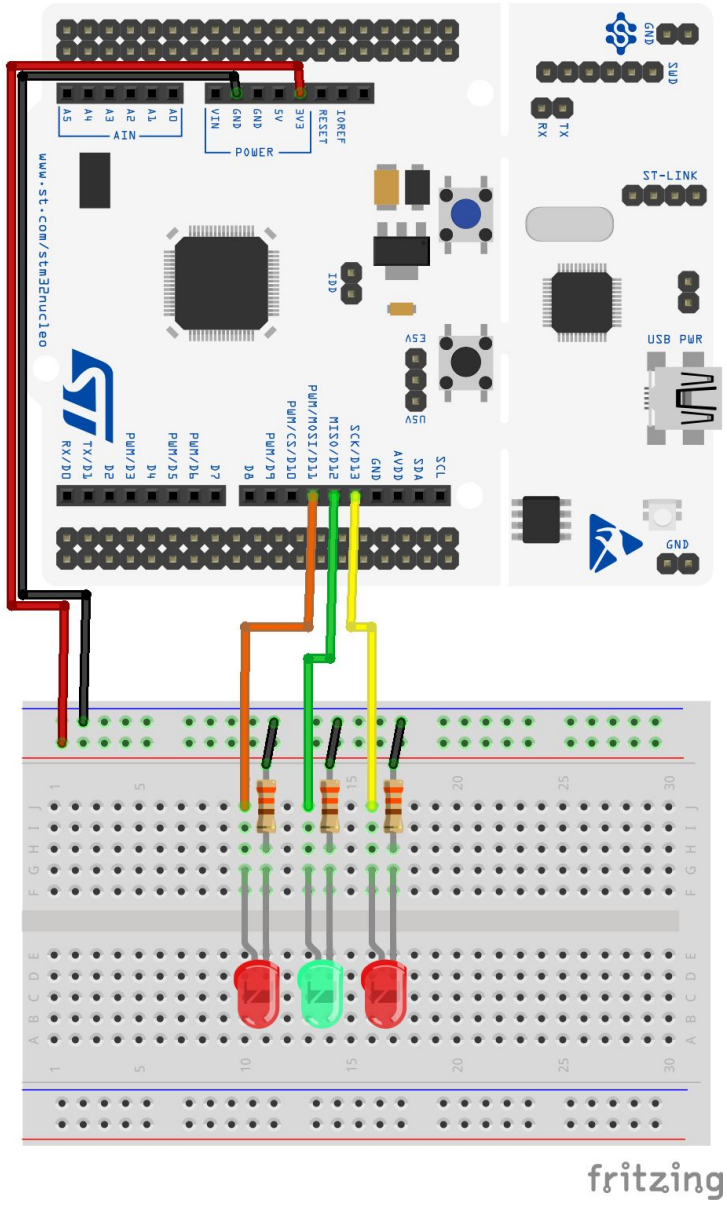
Using the AD2's oscilloscope

Using a pushbutton switch

Lab 1

Wire up three LEDs, as shown. Use both red and green.

1. **blinkySIM.c**: blink all three LEDs simultaneously.
2. **blinkySEQ.c**: blink the LEDs sequentially, so only one LED is lit at a time.
3. **blinkyCNT.c**: blink the LEDs in a binary code: 000, 001, 010,...111, 000...



Code Review

- Share **blinkyCNT.c** in your group.
- Read, explain, understand your partners' code.

Solution blinkySIM.c: blink all three LEDs simultaneously

```
#include "ES28.h"
#define GPIOAEN          (1U<<0)         // bit 0 enables clock for GPIOA
// LEDs on D13, D12, D11 (PA 5, 6, 7)
#define LED_PIN1         (1U<<5)         // Bit 5 of Port A is wired to the LED1
#define LED_PIN2         (1U<<6)         // Bit 6 of Port A is wired to the LED2
#define LED_PIN3         (1U<<7)         // Bit 6 of Port A is wired to the LED3
#define LED_ALL          (LED_PIN1 | LED_PIN2 | LED_PIN3)
int main(void) {
    RCC->IOPENR |= GPIOAEN;             // 1. Enable clock access to GPIOA

    // 2. Configure PA5, 6, 7 as output pin
    GPIOA->MODER |= (1U<<10) | (1U<<12) | (1U<< 14);
    GPIOA->MODER &= ~(1U<<11) & ~(1U<<13) & ~(1U<<15);

    while(1) {
        GPIOA->ODR |= LED_ALL;          // 3. turn LEDs on
        delay_ms(500);                  // kill some time

        GPIOA->ODR &= ~LED_ALL;         // 4. turn LEDs off
        delay_ms(500);                  // kill some time
    }
    return 0;                           // never reached
}
```

Solution blinkySEQ.c: create a "running light"

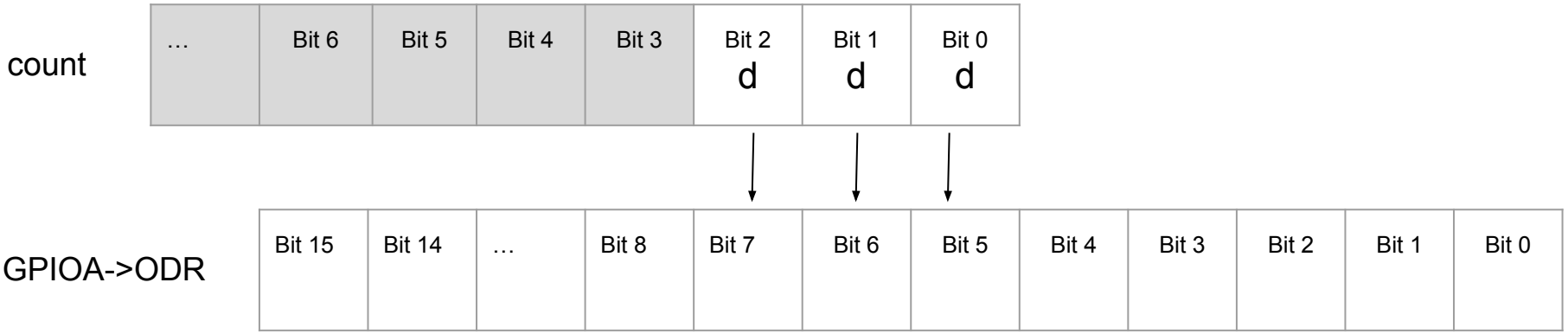
```
#include "ES28.h"
#define GPIOAEN          (1U<<0)         // bit 0 enables clock for GPIOA
// LEDs on D13, D12, D11 (PA 5, 6, 7)
#define LED_PIN1         (1U<<5)         // Bit 5 of Port A is wired to the LED1
#define LED_PIN2         (1U<<6)         // Bit 6 of Port A is wired to the LED2
#define LED_PIN3         (1U<<7)         // Bit 6 of Port A is wired to the LED3

int main(void) {
    RCC->IOPENR |= GPIOAEN;              // 1. Enable clock access to GPIOA

    // 2. Configure PA5, 6, 7 as output pin
    GPIOA->MODER |= (1U<<10) | (1U<<12) | (1U<< 14);
    GPIOA->MODER &= ~(1U<<11) & ~(1U<<13) & ~(1U<<15);

    while(1) {
        GPIOA->ODR = (GPIOA->ODR & ~LED_PIN3) | LED_PIN1; // switch to LED1
        delay_ms(500);                   // kill some time

        GPIOA->ODR = (GPIOA->ODR & ~LED_PIN1) | LED_PIN2; // switch to LED2
        delay_ms(500);                   // kill some time

        GPIOA->ODR = (GPIOA->ODR & ~LED_PIN2) | LED_PIN3; // switch to LED3
        delay_ms(500);                   // kill some time
    }
    return 0;                        // never reached
}
```

blinkyCNT.c

To turn an individual output bit n of Port x on and off:

```
GPIOx->ODR |=  (1U << n);
GPIOx->ODR &= ~(1U << n);
```

How to turn multiple bits on and off? In particular, how do you display the binary count?



One possibility: Turn all bits off, then turn on those that you wish to turn on:

```
GPIOx->ODR &= ~( (1U << 5) | (1U << 6) | (1U << 7));
GPIOx->ODR |= (count << 5);
```

Note: Turning off all bits can also be accomplished this way (**WHY?**):

```
GPIOx->ODR &= ~(0x7 << 5);
```

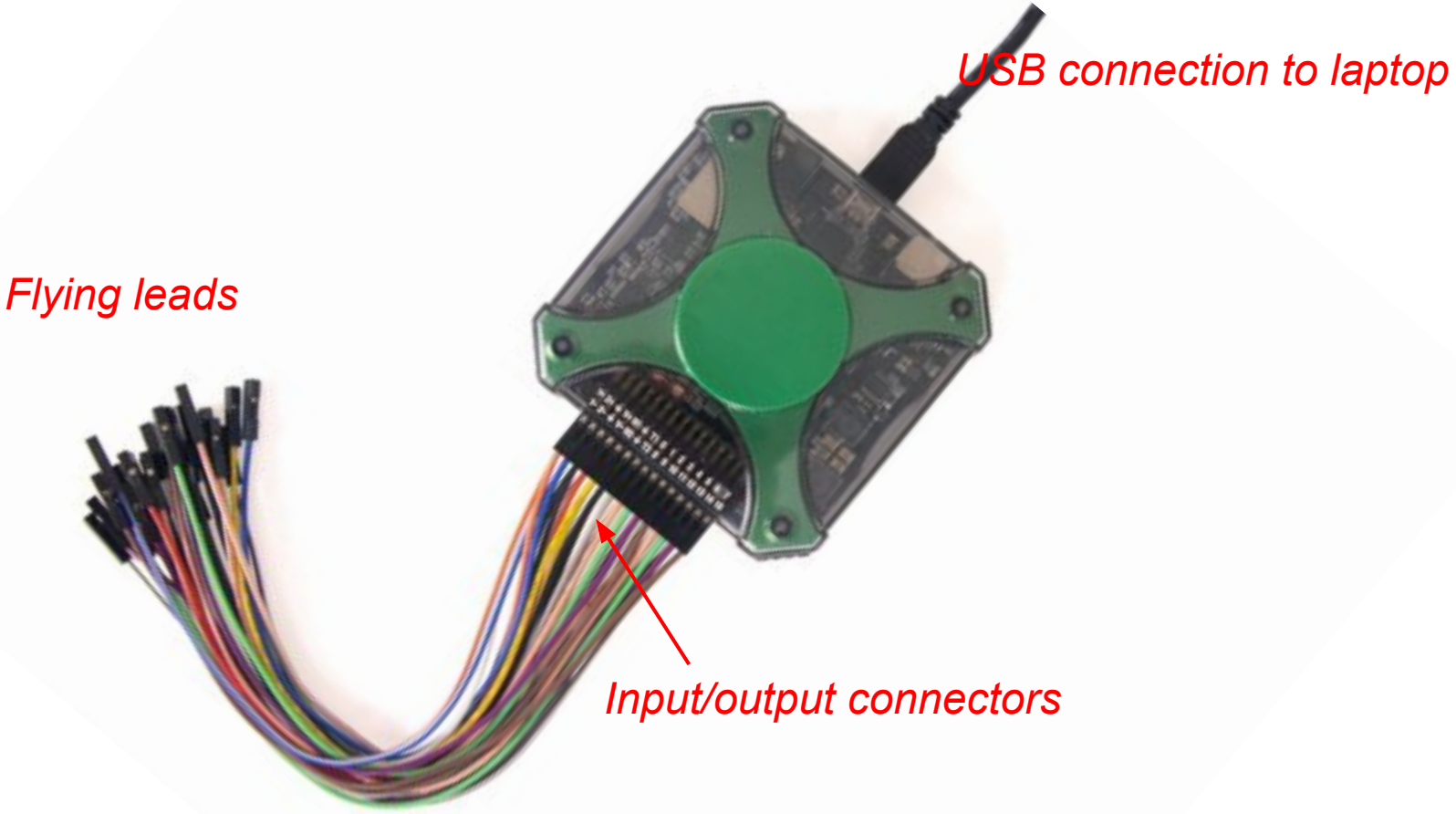
Solution blinkyCNT.c: create a counter

```
#include "ES28.h"
#define GPIOAEN         (1U<<0)         // bit 0 enables clock for GPIOA
// LEDs on D13, D12, D11 (PA 5, 6, 7)
#define LED1_BIT         5
#define LED_PIN1         (1U<<LED1_BIT)             // Bit 5 of Port A is wired to the LED1
#define LED_PIN2         (1U<<(LED1_BIT+1))         // Bit 6 of Port A is wired to the LED2
#define LED_PIN3         (1U<<(LED1_BIT+2))         // Bit 7 of Port A is wired to the LED3
#define LED_ALL          (LED_PIN1 | LED_PIN2 | LED_PIN3)

int main(void) {
    RCC->IOPENR |= GPIOAEN;                        // Enable clock access to GPIOA

    // Configure PA5, 6, 7 as output pin
    GPIOA->MODER |= (1U<<10) | (1U<<12) | (1U<< 14);
    GPIOA->MODER &= ~(1U<<11) & ~(1U<<13) & ~(1U<<15);

    while(1) {
        for (unsigned count = 0; count<8; count++){
            GPIOA->ODR &= ~(LED_ALL);              //turn all LEDs off
            GPIOA->ODR |= count<<LED1_BIT;
            delay_ms(500);                         // kill some time
        }
    }
    return 0;                                      // never reached
}
```

Watching signals in time (oscilloscope)

Making measurements: Analog Discovery 2

"AD2" contains oscilloscope, waveform generator, and other test tools.



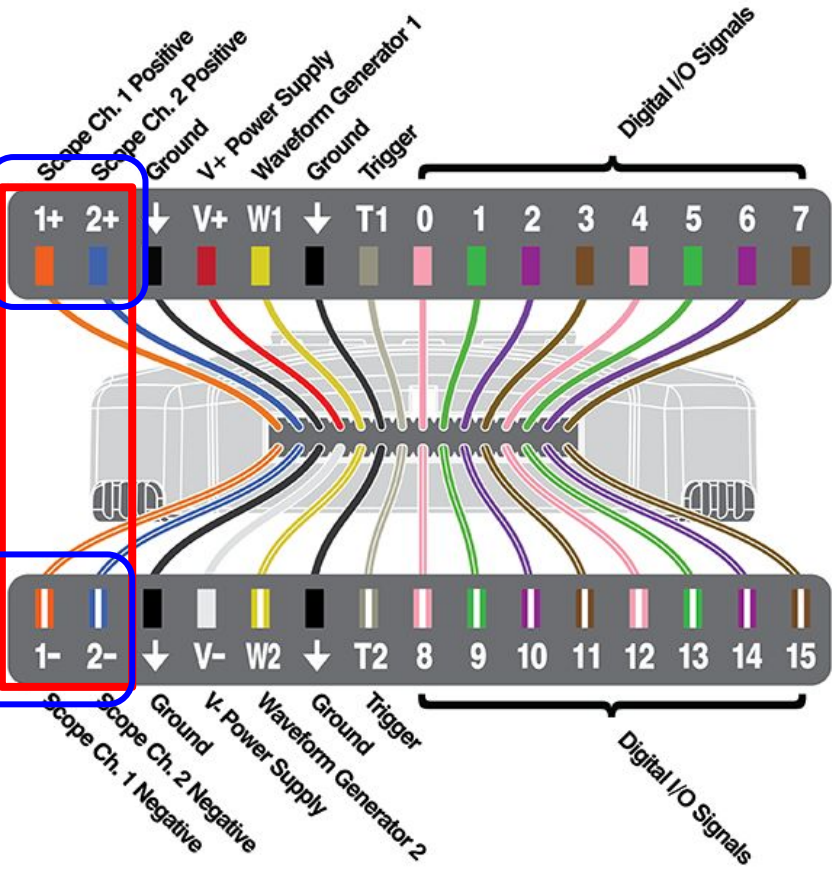
AD2 connector pinout

Use “flying leads” to connect AD2 to Nucleo and/or breadboard.

Plus leads are solid colored

Two analog input channels

Minus leads have a white stripe on them

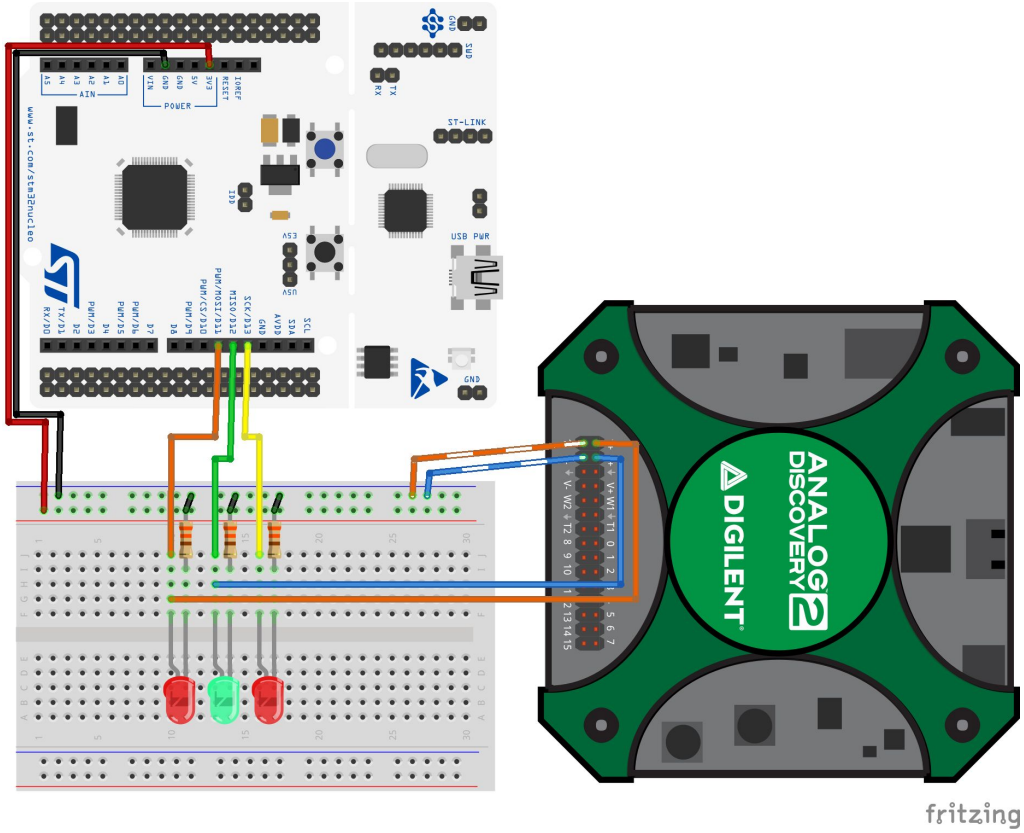


We’ll get to the other I/O channels later

Wire the AD2 to your Lab 1 setup

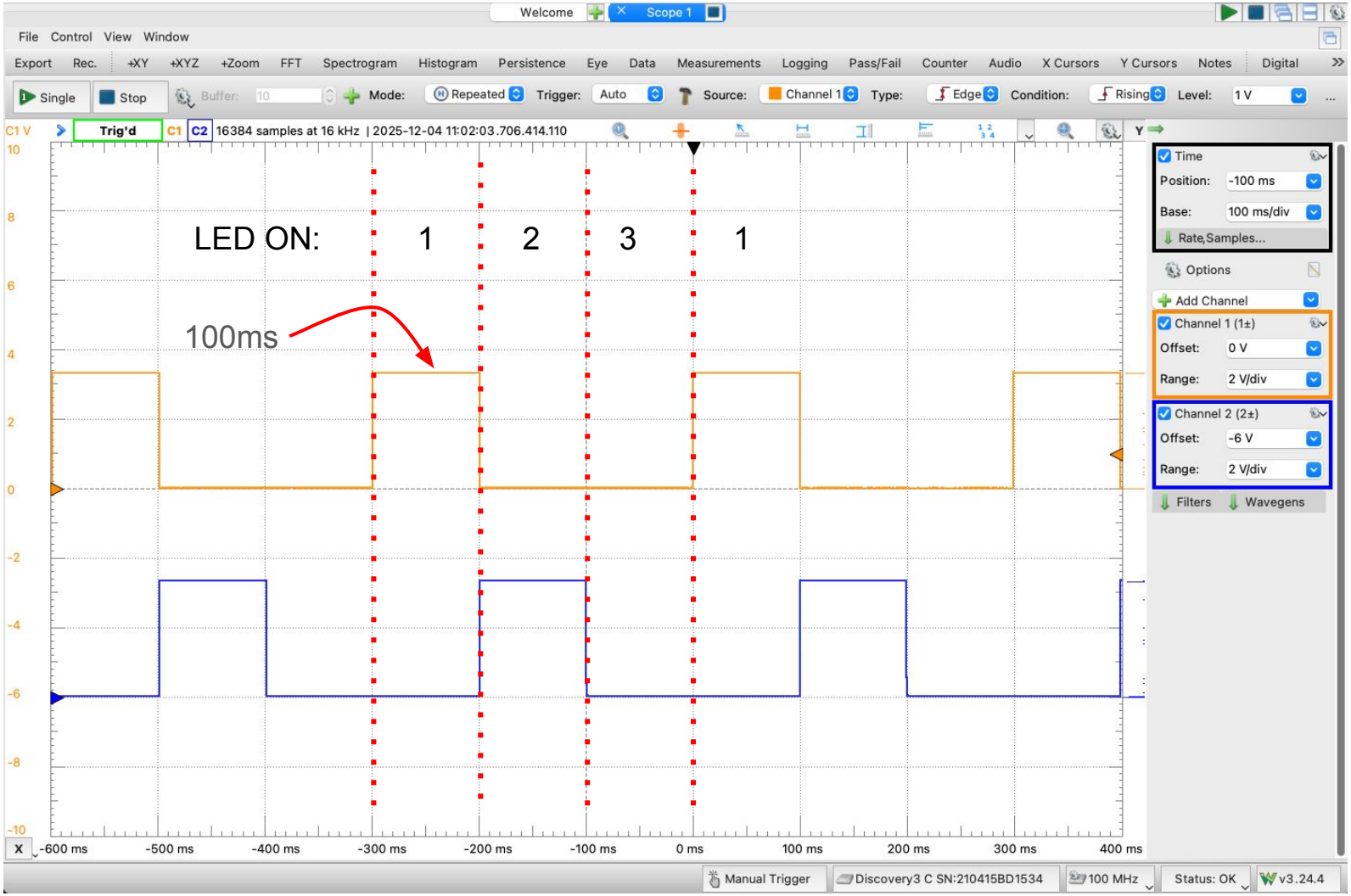
Load BlinkySEQ onto your Nucleo.

Use “flying leads” to connect AD2 to your breadboard: “–” leads to ground, “+” leads to the LED anodes.



Open Waveforms, select "Scope".

Observe two channels with Waveforms



Waveforms Settings

Experiment with different settings for Time Base, Range, Offset, both for CH1 and CH2. Answer the following questions as a table group:

- What does "Time Base" do?
- What does Range do?
- What does Offset do?
- How do you change which channel's voltage is shown on the vertical axis?
- What is the voltage on an output pin when it is high?
- Save as PNG, and see what you get. (You'll need this for labs going forward)
    - File->Export
    - Select the "Image" tab
    - Click "Save"

One more experiment

Edit **blinkySIM.c**:

1. Change the blink rate as follows and note the brightness of the LEDs and observe the waveform.

```
while(1) {
    GPIOA->ODR |= LED_ALL;      // 3. turn LEDs on
    delay_ms(1);                // kill some time

    GPIOA->ODR &= ~LED_ALL;     // 4. turn LEDs
off
    delay_ms(10);               // kill some time
}
```

2. Now swap the lengths of on and off time; again note the brightness and the waveform.

```
while(1) {
    GPIOA->ODR |= LED_ALL;      // 3. turn LEDs on
    delay_ms(10);               // kill some time

    GPIOA->ODR &= ~LED_ALL;     // 4. turn LEDs
off
    delay_ms(1);                // kill some time
}
```

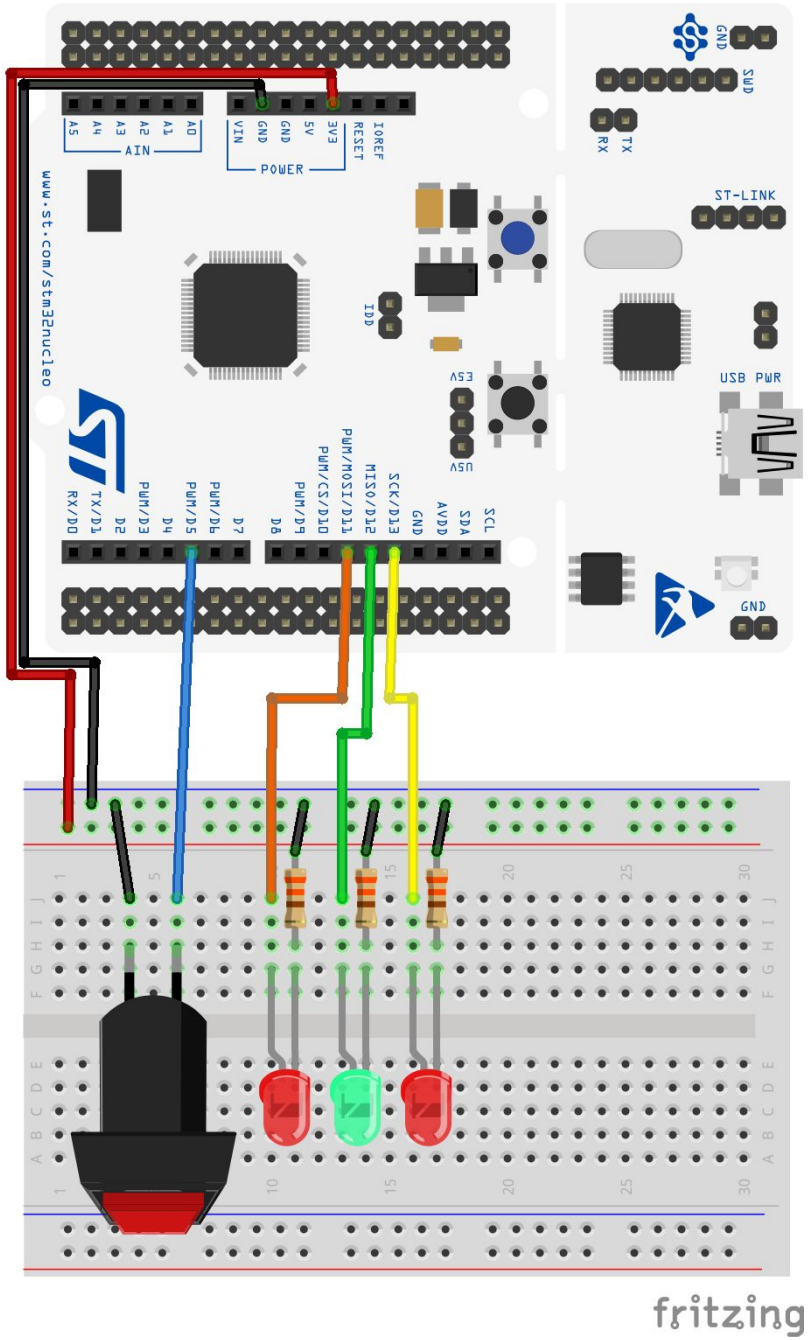
The oscilloscope can see the on/off in the LED signal, even when your eyes cannot.

Discuss at your table why the LED brightness changes. How could this be useful?

Using pins for INPUT

Wire up the next circuit

Add a pushbutton switch to your Lab 1 breadboard, like this.



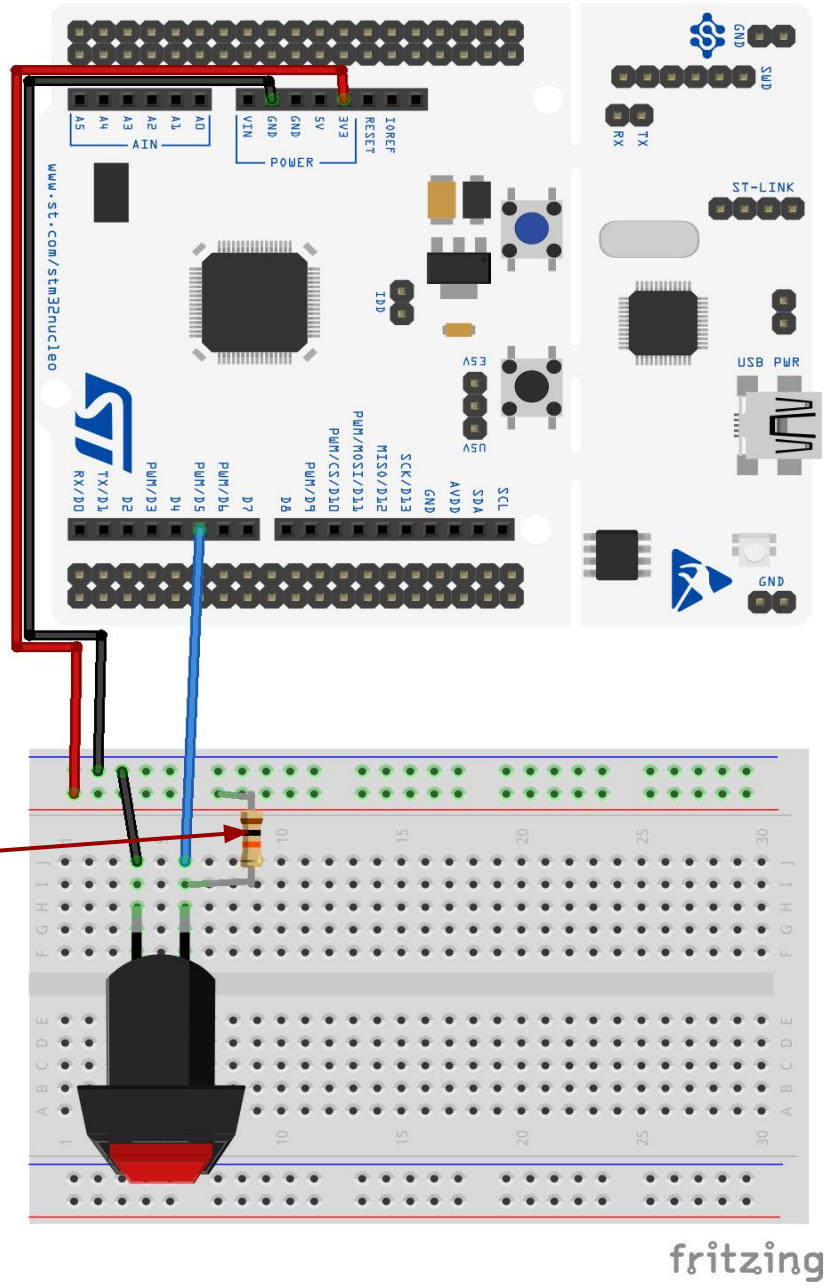
How to wire a pushbutton

A button by itself does not produce a 1 or a 0 — it simply closes a connection from the pin to ground.

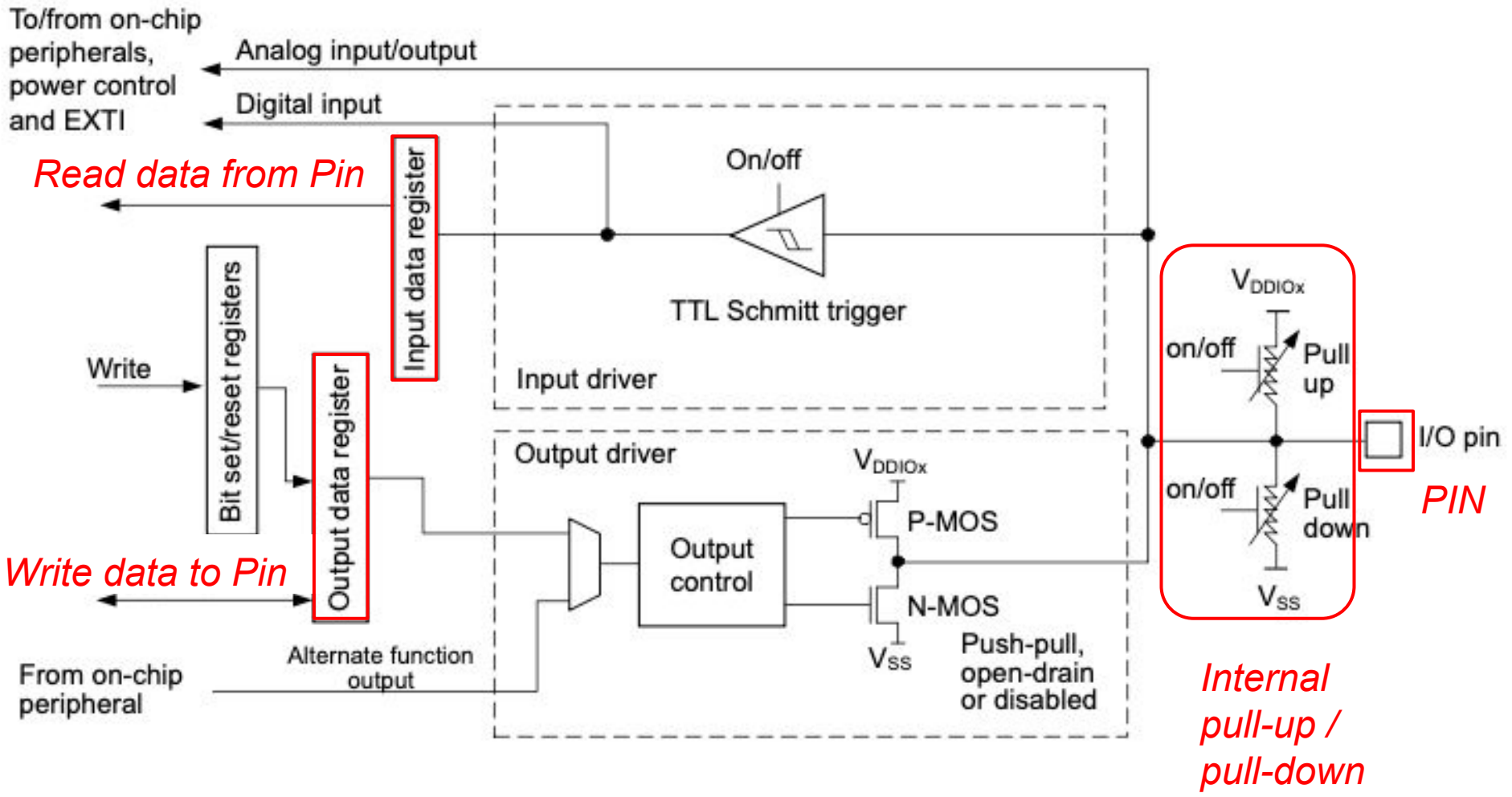
“Pull the pin up” so that it is HIGH when the switch is unpressed. Two possibilities:

- An external resistor (about 10K)
- Enable the pin’s *internal* pullup resistor

Pressing the button connects the pin to ground and pulls the pin LOW. (“Low true”)



Basic structure of an I/O port bit

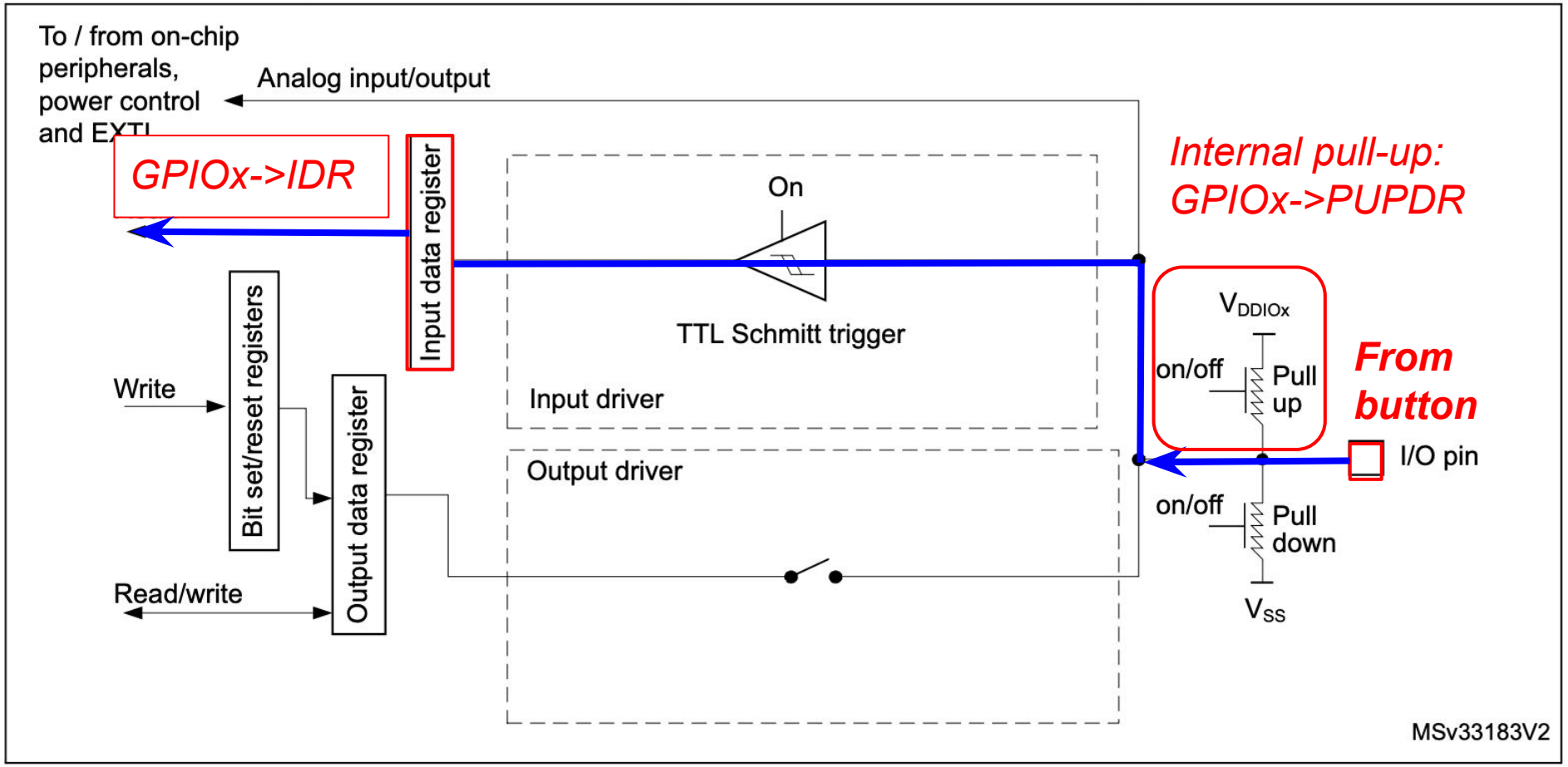


Input pin configuration

x = Port (A, B, C, D, F)

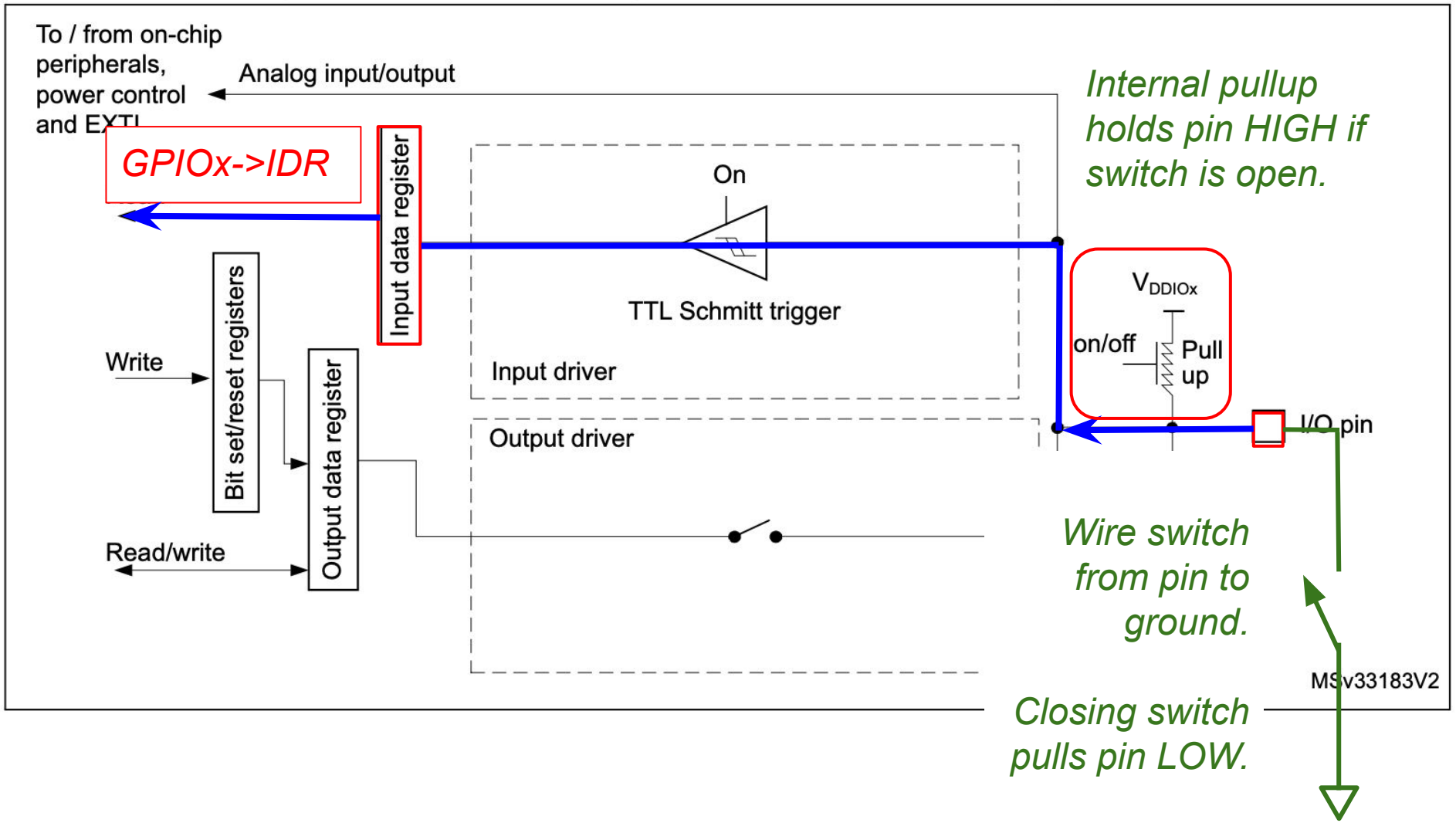
GPIOx->MODER determines input versus output mode, amongst others.

Figure 15. Input floating/pull up/pull down configurations



How to wire a pushbutton

Figure 15. Input floating/pull up/pull down configurations



How to use a pin as INPUT

- GPIOx->MODER = "Mode Register for GPIO Port x", contains 32 bits: two for each pin to determine its mode: Output, input, analog I/O, etc
- Bits 2n and 2n+1 configure pin n.

configure pin 15 configure pin 6 configure pin 5 configure pin 0

Bit 31	Bit 30	…	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	…	Bit 2	Bit 1	Bit 0
1	0		1	1	0	1				0	0

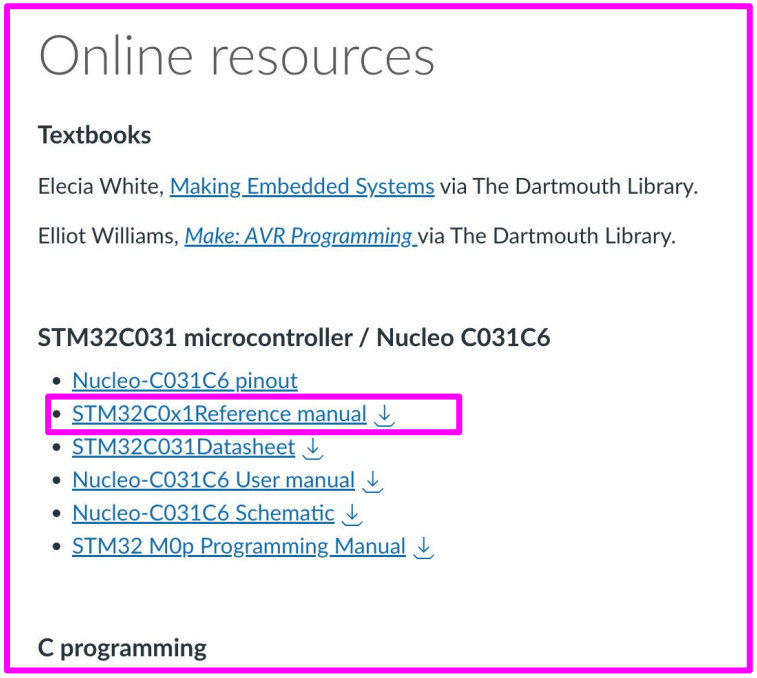
In this **example**:

- pin 0 can be used as an input pin (ex: pushbutton)
- pin 5 as an output pin (ex: LED)
- pin 6 as an analog pin (ex: analog temperature sensor)
- pin 15 in alternate function mode (ex: control motor speed)

By default (after a reboot) all I/O lines start as inputs (00).

Bits	Function
00	Input
01	Output
10	Alternate function
11	Analog

How do you know???



The **Reference Manual** contains all of this information!

- Open the reference manual.
- Scroll through the table of contents.
- Find the General-purpose I/Os (GPIO) entry, follow the link to the GPIO section.
- Scroll through the section and note where the images for input/output function were taken from!
- Go to Section 6.4 GPIO registers
- GPIOx_MODER is described in 6.4.1

Note: GPIOx_MODER is the name of the register. GPIOx->MODER is a dereferenced pointer to the memory location that is mapped to the register. To change GPIOx_MODER we write to GPIOx->MODER. So in our code we'll always use GPIOx->MODER.

Enabling the Internal Pullup Resistor

- Each GPIO pin has an optional internal pull-up resistor associated with it.
  - If the pull-up is enabled, in the absence of an input signal, the input pin will be "pulled up" to a logical 1.
  - If an active signal is attached to the pin the pull-up has no effect.
- To enable a pull-up resistor for a pin that is used as input (bits in GPIO->MODERx set to '00') we configure the corresponding GPIO port pull-up/pull-down register.

Look at the reference manual and find:

- The name of the pull-up/pull-down register for GPIO port x (x=A,B,...)
- How do we refer to it in code?
- Which bits should we set to what values in order to configure pin 4 of port B as an input pin?

Enabling the Internal Pullup Resistor

- The name of the pull-up/pull-down register for GPIO port x:
  GPIOx_PUPDR
- How do we refer to it in code?
  GPIOx->PUPDR
- Which bits should we set to what values in order to configure pin 4 of port B as an input pin?
  Bit 8 of GPIOB->PUPDR: 1
  Bit 9 of GPIOB->PUPDR: 0

Bits 31:0 **PUPDy[1:0]:** Port x configuration I/O y (y = 15 to 0)
These bits are written by software to configure the I/O
00: No pull-up, pull-down
01: Pull-up
10: Pull-down
11: Reserved

6.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A, B, C, D, F)

Address offset: 0x0C

Reset value: 0x2400 0000 (for port A)

Reset value: 0x0000 0000 (ports other than A)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Reading the state of a pin

To read the state of pin n in PORTx (x = A, B, C, D, F):

- Ensure that bits 2n and 2n+1 in GPIOx->MODER are cleared (input).
- If desired, enable internal pull-up resistor by setting bits 2n and 2n+1 in GPIOx->PUPDR to 01.
- Read bit n of GPIOx->IDR (input data register). Read all 16 bits in the register, perform logical AND with a bit mask to isolate pin n.

Example: Button wired to Nucleo pin D5 (PB4):

Setup:

```
// Configure PB4 as input pin
GPIOB->MODER &= ~(1U<<8);
GPIOB->MODER &= ~(1U<<9);
// (may not be necessary since
this is default reset value)

// enable pull-up for PB4
GPIOB->PUPDR |= (1U<<8);
GPIOB->PUPDR &= ~(1U<<9);
```

Read button state:

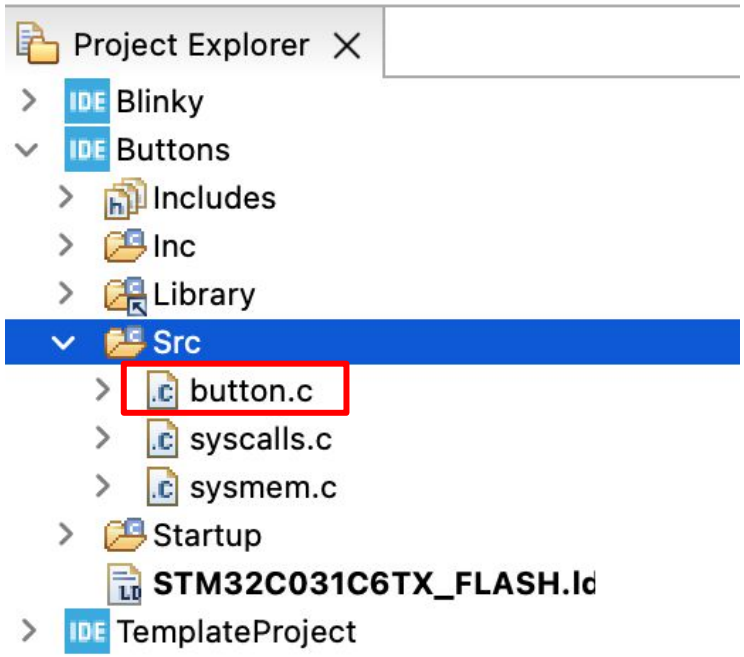
```
// Button on PB4 (D5)
#define BUTTON_PIN (1U<<4)
…
if ((GPIOB->IDR & BUTTON_PIN) == 0) {
      // button pressed
      // do stuff
} else {
      // button not pressed
      // do other stuff
}
```

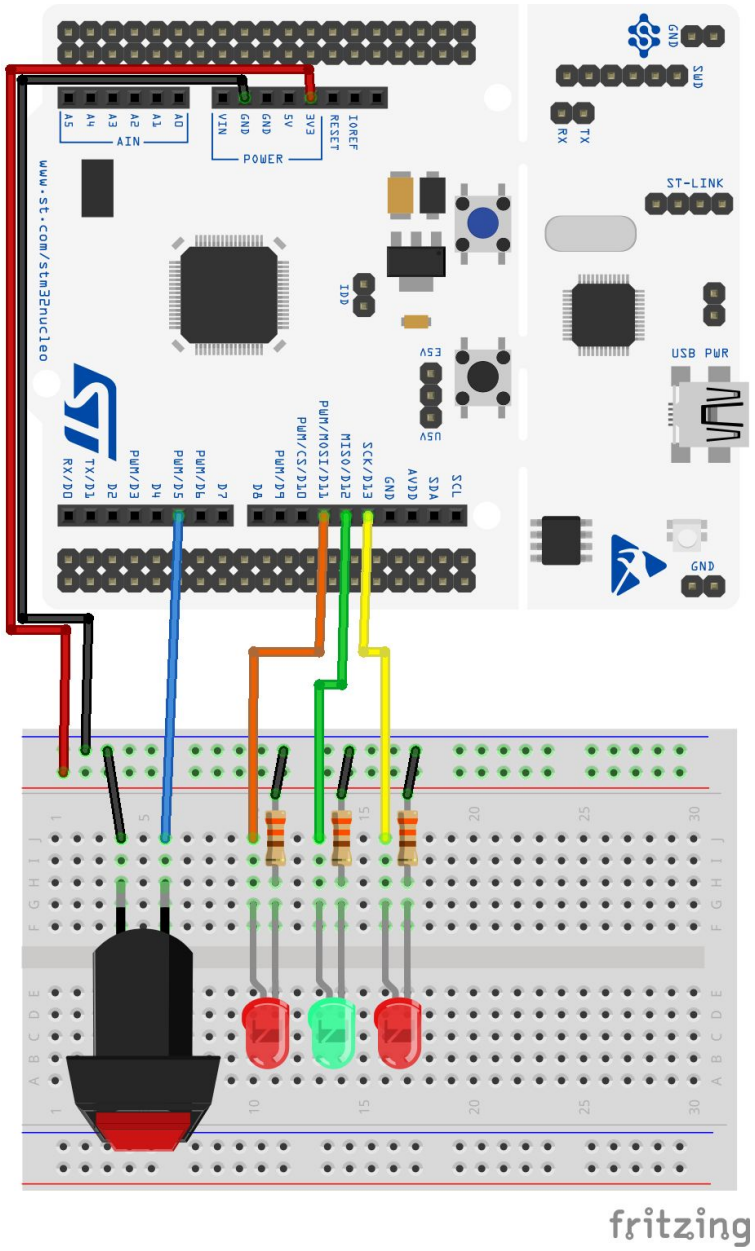
Pushbutton exercises

Create a new copy of your TemplateProject and name it "Buttons". This creates an entirely new project for you.

Download “button.c” from Canvas and place it into the Src folder in your buttons project.

This is how things should look in your Project Explorer in the IDE:





button.c

```
// Press button to light the built-in LED
#include "ES28.h"                 // All the port definitions are here

#define GPIOAEN         (1U<<0)         // Clock access to GPIOA (LED)
#define GPIOBEN         (1U<<1)         // Clock access to GPIOB (button)

#define LED_PIN         (1U<<5)         // LED on PA5      (D13)
#define BUTTON_PIN      (1U<<4)         // Button on PB4 (D5)

int main(void) {
    RCC->IOPENR |= (GPIOAEN | GPIOBEN);   // Enable clock access to GPIOA and B

    GPIOA->MODER |= (1U<<10);             // Configure PA5 as output pin
    GPIOA->MODER &= ~(1U<<11);

    GPIOB->MODER &= ~(1U<<8);             // Configure PB4 as input pin
    GPIOB->MODER &= ~(1U<<9);             // (default is 0b11)

    GPIOB->PUPDR |= (1U<<8);              // enable pull-up for PB4
    GPIOB->PUPDR &= ~(1U<<9);
    delay_ms(50);                         // give pull-ups time to pull up the pins

    while(1) {
        // Read the pin and control the LED
        if ((GPIOB->IDR & BUTTON_PIN) == 0) {    // button input is low-true
            GPIOA->ODR |= LED_PIN;               // turn LED on
        } else {
            GPIOA->ODR &= ~LED_PIN;              // turn LED off
        }
    }
    return 0;                                    // never reached
}
```

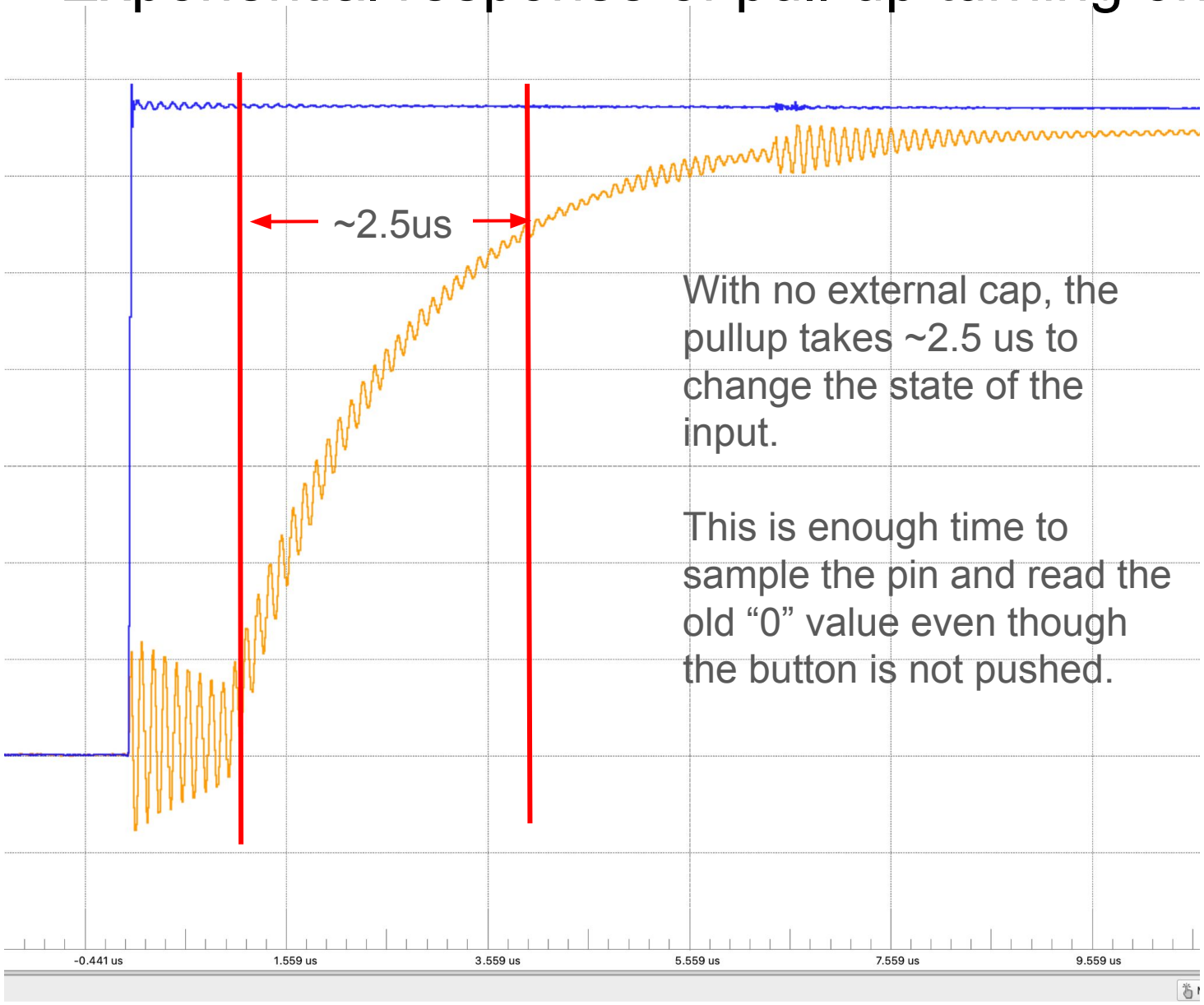
Why the delay_ms() after the pull-ups?

```
// Press button to light the built-in LED
#include "ES28.h"                  // All the port definitions are here

#define GPIOAEN         (1U<<0)         // Clock access to GPIOA (LED)
#define GPIOBEN         (1U<<1)         // Clock access to GPIOB (button)

#define LED_PIN         (1U<<5)         // LED on PA5      (D13)
#define BUTTON_PIN      (1U<<4)         // Button on PB4 (D5)

int main(void) {
    RCC->IOPENR |= (GPIOAEN | GPIOBEN);   // Enable clock access to GPIOA and B

    GPIOA->MODER |= (1U<<10);             // Configure PA5 as output pin
    GPIOA->MODER &= ~(1U<<11);

    GPIOB->MODER &= ~(1U<<8);             // Configure PB4 as input pin
    GPIOB->MODER &= ~(1U<<9);             // (default is 0b11)

    GPIOB->PUPDR |= (1U<<8);
    GPIOB->PUPDR &= ~(1U<<9);
    delay_ms(50);                                              ll up the pins

    while(1) {
        // Read the pin and control
        if ((GPIOB->IDR & BUTTON_PI                            -true
            GPIOA->ODR |= LED_PIN;
        …
```

This code takes about 1.5us to execute.

How long does the pull-up take to get to a logical 1?

Exponential response of pull-up turning on.

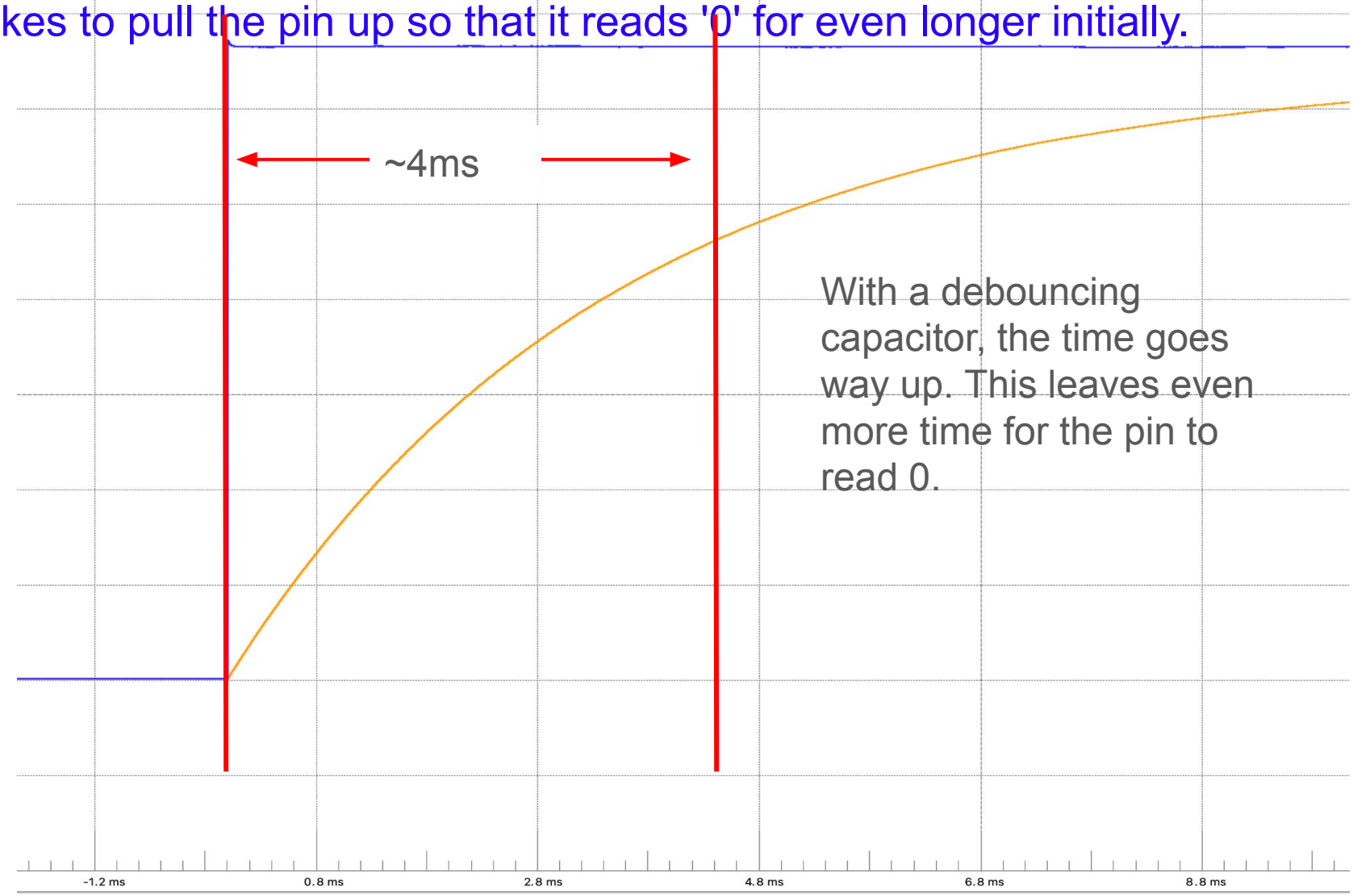


With no external cap, the pullup takes ~2.5 us to change the state of the input.

This is enough time to sample the pin and read the old “0” value even though the button is not pushed.

Exponential response with 0.1uF debouncing cap.

Tomorrow we'll be adding a capacitor to our setup to improve the button's behavior. This actually makes things worse when it comes to the time it takes to pull the pin up so that it reads '0' for even longer initially.



Fundamental Issue: Code executes super fast but it takes time for the pins to get to the '1' state.

- In general the pins on the chip start out “0”.
- Debouncing makes it take longer for the pullup to make them “1”.

How can we fix it?

- Easiest:
  - Wait (delay_ms) 50ms before sampling any pin - add a delay after initialization.
- Others:
  - Loop until the pin reads “1”
  - Start as an output, set the pin to a “1”, then switch modes and enable the pullup.
  - Use an *external* pullup.

Caution!

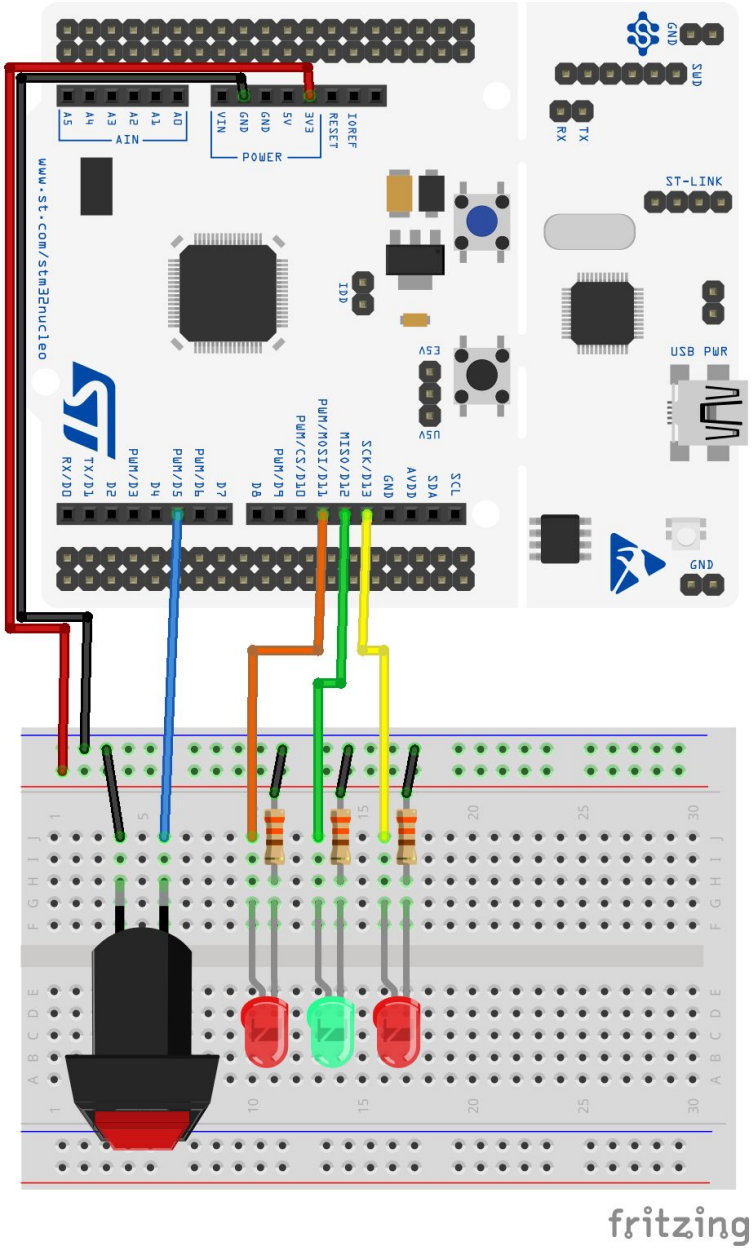
This will happen any time the pin is pulled down. Your code needs to account for this! Best way in “regular” code is to use a state machine. Make sure you transition from PRESSED to UNPRESSED before taking another action.

Pushbutton exercises

Make a copy of one of your Lab 1 programs and modify them to use the button:

- **blinkySEQ.c**: Pressing the button turns all the LEDs on, releasing it returns to the usual sequence.
- **blinkyCNT.c**: A button press causes the counter to pause, a second press causes it to resume.

(This is part of your Lab 2 work. You won't quite get this done right now but can get a nice headstart.)



Homework for Thursday

Write code (toggleLED.c) so that the **on-board LED** is toggled on and off via the button:

PRESS => LED goes ON
RELEASE => LED stays ON
PRESS => LED goes OFF
RELEASE => LED stays OFF

You may observe “glitches” (erratic behavior of the LED). We’ll investigate on Thursday.